General disclosure

For this assignment, I used Anthropic Claude Opus 5.5 (claude-opus-5-5) through the Claude app. I used Claude on September 26–27, 2026.

Code attribution

Prompt 1: Write a C++ program that can find a valid IPv4 address inside of a line of text. The address might also have a port number after it. For the IPv4 address, only numbers, periods, and colons can be part of it. Everything else in the string can basically be treated as a separator or garbage and skipped. An IPv4 address needs to have exactly four numbers separated by periods. Each number can only be between 0 and 255 and can have 1–3 digits. Don't allow leading zeros unless the number is just 0. The address can also have a port after it. The port can be 1–5 digits and has to be between 0 and 65535. The same leading zero rule applies to the port. If there is a colon but the port is invalid, reject the entire address. Make sure you check the entire possible address instead of just finding a valid part of it. For example, "192.168.1.1." should be invalid because of the extra period at the end. If there are multiple possible addresses in the string, return the first valid one you find. If there isn't a valid address, set "outAddress" to 0 and "outPort" to -1. Don't use built-in functions that do the parsing for you, including "atoi", "stoi, "strtol", "sscanf", "inet_pton", or regex. Build the numbers yourself one digit at a time. You can use "isdigit". Also make sure really long numbers don't cause an overflow. For "main", keep asking the user to enter a string until they type exactly "END". When they type "END", print "Program terminated." If an address is found, print it like this: "Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)"If there is no port, print "none" for the port.

Prompt 2: Can you make a test program for this with at least 40 different test cases? Make sure you test the normal cases but also weird edge cases.

Prompt 3: Can you go back through the code you wrote and compare it to all of the requirements I gave you? Go through each requirement and explain where the code handles it. Also try to find any inputs or edge cases that could still break the code or give me the wrong answer.

Prompt 4: Can you specifically check 1.2.3.4:080? The assignment says a port can't have a leading zero unless it is just 0. Make sure the code rejects this and explain what part of the code handles it.

All of the code for this assignment was generated by Claude. My part of the process was coming up with the prompts and reviewing the code that Claude generated. I also tested it to make sure it actually followed the requirements. I also went through the code so I could understand what each part was doing instead of just assuming it was correct because it was generated by AI. I did not manually make any major changes to the code that Claude generated. Instead, I used follow-up prompts to have Claude review its work and check specific things I thought could cause

problems. I used follow-up prompts whenever there was something in the code I wanted to double-check. This helped me make sure the code actually followed the assignment requirements and handled the different edge cases correctly. After testing the final version, I did not find anything that needed to be manually changed. Because the code passed the test cases and behaved correctly during my manual testing, I kept the AI-generated code as it was.

Verification Statement

Even though the code was generated by AI, I made sure I understood how it worked before submitting it. I went through the code and made sure I understood why it was written the way it was and how it followed the requirements of the assignment. I also compiled and ran the program myself instead of relying on Claude to tell me that it worked. I ran all 46 test cases and also tried my own inputs to see how the program handled them. Everything worked as expected during my testing. I am not aware of any bugs in the final program.